

SYSTEMS DESIGN · REINFORCEMENT LEARNING · COMPUTER VISION · NLP · SOFT COMPUTING

NexusSim

Explained, Engineered — a real-city, real-time multi-agent traffic simulator, and its integrated extension into an AI-driven policy sandbox.

ENGINE	PIPELINE	EXTENSION	OBJECTIVE
C++17 · Quadtree · SoA	Python · OSM + real OD demand	RL · CV · NLP · GA	Speed + Equity, not just speed

Revision 2. This edition rebuilds every table in the document for readability (numbered captions, grouped and shaded rows, consistent units) and rewrites the surrounding explanations in plain language, with technical terms defined. No underlying data values were changed — only how they are presented and explained. Section order and layout follow revision 1 exactly.

Table of Contents

1. Overview	3
Table 1 · Who NexusSim is built for · Table 2 · Plain-language glossary	
2. Architecture	5
Table 3 · Where each part of the system lives · Table 4 · How data moves	
3. Explanation — How NexusSim Works	8
Table 5 · The five core algorithms	
4. Engineering Design	11
Table 6 · Key engineering decisions · Table 7 · Performance targets · Table 8 · Scaling	
5. Examples & Use Cases	15
Table 9, 10, 11 · Three worked walkthroughs	

How to read this document. Each table is numbered and titled, introduced by a sentence explaining what it shows, and followed by a green *Key takeaways* box highlighting the most important points. Every page carries its number in the footer.

Assumption noted: this document has a mixed audience (see Table 1) that includes technical reviewers, so the tables retain precise file paths and technical terms. All explanations around them are written for a non-technical reader, and Table 2 defines every technical term used.

Overview

Purpose

NexusSim is a high-fidelity, real-city traffic simulation platform. Its purpose is to let urban planners and researchers test policy changes — a new bus lane, a congestion-pricing zone, a different traffic-signal strategy — inside a realistic, physics-driven copy of a city before committing to them in the real world. A second, equally central purpose is to treat **equity** as a first-class, measured output: a policy that speeds up the average commute while quietly leaving poorer neighborhoods stuck is treated as a failure, not a success.

Core functionality

- Loads a real city's road network from OpenStreetMap and simulates thousands of individually modeled vehicles and pedestrians moving through it, tick by tick, in real time.
- Streams live simulation state to a browser dashboard over WebSocket, rendered on an interactive map with efficiency and equity views.
- Calibrates itself against real trip data (currently Chicago's Transportation Network Provider data) and reports a quantitative accuracy checkpoint, not just a qualitative “looks right.”
- Is being extended into an integrated AI platform: adaptive (reinforcement-learned) traffic signals, computer-vision-based congestion sensing, a natural-language interface to live metrics and incident reporting, and genetic-algorithm-based calibration — all wired through the simulation's existing live data stream rather than as separate demos.

Intended audience

NexusSim serves three different kinds of reader, each of whom uses a different part of it. The table below matches each reader to what the system actually gives them — useful for deciding which sections of this document matter to you.

Table 1: Who NexusSim Is Built For

Reader	Primary interface they use	What they get from NexusSim
Urban policy researcher / student developer	The full codebase	Full technical access: builds the system, interprets results, writes reports.
Civil administrator / policy audience	The dashboard only	Plain-language metrics, policy toggles, no jargon required.
Technical reviewer / evaluator	Documentation and source	Documented interfaces, cited design decisions, and validation methodology — this document is written primarily for this audience.

KEY TAKEAWAYS

- Only one of the three audiences needs technical skill to get value — the administrator uses the dashboard alone.
- This document targets the third row (technical reviewer), which is why the tables that follow keep precise file paths.

Source: audience definitions carried forward from docs/PRD.md §4, verified against this document's own scope. The "Primary interface" column restates information already given in the original descriptions; no new facts were introduced.

Plain-language glossary

The tables in this document use a handful of technical terms. Each is defined once here, in everyday language, so the rest of the document can be read without a computing background.

Table 2: Technical Terms Used in This Document, Defined

Term	What it means in plain language
Agent	One simulated traveller — a car, bus, rickshaw, two-wheeler, or pedestrian.
Tick	One step of the simulation clock. NexusSim runs one tick every 0.1 seconds of simulated time.
Graph (road graph)	The road network stored as a set of junctions and the road segments connecting them.
Origin–destination (OD) demand	A count of how many real trips go from each area to each other area, by hour of day.
Gini coefficient	A single fairness score from 0 to 1, borrowed from economics. 0 means every zone waits equally; higher numbers mean waiting is concentrated in some zones.
MAPE (mean absolute percentage error)	How far the simulation's predicted journey times sit from real measured ones, on average, as a percentage. Lower is better.
WebSocket	A continuously open connection that lets the simulation push live updates to the browser.
Quadtree	An index that makes "which vehicles are near this point?" fast to answer.
ONNX	A portable file format for a trained AI model, so it can run outside the language it was trained in.
Reinforcement learning (RL)	Training a decision-maker by trial and error against a score, rather than programming fixed rules.

KEY TAKEAWAYS

- Every term above appears in a later table; none require prior technical knowledge once defined.
- Two terms — **Gini coefficient** and **MAPE** — are the document's two headline quality measures: one for fairness, one for realism.

Architecture

2.1 High-level system design

NexusSim is three cooperating subsystems today, each behind a documented interface so it can be built, tested, and handed off independently. Reading left to right below is also the order data travels: raw map data becomes a clean road network, the engine simulates traffic on it, and the dashboard shows the result to a person.

1. DATA PIPELINE Python 3.11 · OSM cleaning, lane inference, zone tagging, OD demand → graph.json + od_matrix.json	2. SIMULATION ENGINE C++17 · IDM car-following, A* routing, quadtree spatial index, Webster's-formula signals → live tick loop	3. DASHBOARD React 19 + TypeScript + Leaflet · live map, efficiency/equity metrics → human-facing surface
---	--	---

Handoff 1 → 2: graph.json / od_matrix.json | Handoff 2 → 3: WebSocket JSON (agents, metrics, zone_metrics)

2.2 Main components

This table maps each named part of the system to the file it lives in and the job it does. It matters because it is the fastest way for a reviewer to find the code behind any claim in this document. Rows are grouped by which subsystem they belong to.

Table 3: Where Each Part of the System Lives

Component	Source file(s)	Responsibility
SIMULATION ENGINE — C++17 (8 components)		
Graph loader	engine/src/graph/Graph.h, GraphLoader.cpp	Loads the road network (nodes, edges, lanes, zones) from graph.json
Agent system	engine/src/agent/Agent.h	Structure-of-Arrays store for every vehicle/pedestrian's state
Car-following model	engine/src/agent/IDM.h	Per-agent-type acceleration physics (Intelligent Driver Model)
Pathfinder	engine/src/agent/Pathfinder.h	A* routing, plus a stochastic variant for route diversity
Spatial index	engine/src/spatial/Quadtree.h	O(log n) neighbor queries for collision avoidance & lane changes
Signal control	engine/src/agent/SignalController.h	Webster's-formula fixed-cycle intersection timing
Simulation orchestrator	engine/src/agent/Simulation.h	Owns the tick loop, spawns agents, computes metrics, broadcasts state
WebSocket server	engine/src/network/WebSocketServer.h	uWebSockets transport; inbound viewport-bounds handling
DASHBOARD — React 19 + TypeScript (2 components)		

Component	Source file(s)	Responsibility
Dashboard hook	dashboard/src/hooks/useWebSocket.ts	Client-side WebSocket connection, auto-reconnect, message parsing
Map view	dashboard/src/components/Map.tsx	Leaflet rendering of live agents and road geometry

KEY TAKEAWAYS

- **8 of the 10 components sit in the C++ engine** — that is where all the performance-critical work happens, and where most review effort belongs.
- The dashboard is deliberately thin (2 components): it displays and reconnects, but contains no simulation logic of its own.
- Every component listed maps to exactly one file or file pair, so there is no hidden or undocumented module.

2.3 Data flow

This table follows one city's data from raw download all the way to pixels on a screen. It matters because it shows which work happens once, up front (the top group), and which work repeats continuously while the simulation runs (the bottom group) — the distinction that governs where performance actually matters.

Table 4: How Data Moves Through the System, Stage by Stage

Stage	Input	Output
PREPARATION — runs once per city, offline		
1. OSM cleaning	Raw .osm.pbf per city	Cleaned, largest-component road graph
2. Lane / zone inference	Cleaned graph + OSM tags	Per-edge lane counts (confidence-scored), per-node zone IDs
3. OD demand	Chicago TNP trip data (Socrata)	Hourly origin-destination demand matrix
LIVE SIMULATION — repeats continuously, once started		
4. Engine load	graph.json, od_matrix.json	In-memory graph + OD-driven agent spawner
5. Engine tick	Agent state, signal state	Updated positions, wait times, zone metrics (every 0.1 s)
6. Broadcast	Tick output	JSON frame over WebSocket: agents[], metrics[], zone_metrics[]
7. Dashboard render	WebSocket frame	Live map + efficiency/equity panels
KEY TAKEAWAYS • Only stages 5–7 repeat, and they repeat every 0.1 s — roughly 10 times per second, which is why those three stages carry every performance constraint in Part 4. • Stages 1–3 are slow, one-time, and offline, so they can afford expensive processing such as confidence-scoring every inferred lane count. • Each stage's output is exactly the next stage's input, with no back-channels — apart from one feedback path (the dashboard sending its viewport bounds back to the engine).		

2.4 Planned integrations (four-subject extension)

The approved extension plan (see IMPLEMENTATION_PLAN.md Phases 12–18) adds Reinforcement Learning, Computer Vision, NLP, and Soft Computing without replacing this architecture — it reuses the existing WebSocket protocol and the existing cities.yaml config-driven pattern as the integration surface. The three boxes below are grouped by how time-sensitive each addition is.

ENGINE — real-time Pluggable signal policy: Webster's / Fuzzy / RL-via-ONNX, hot-swappable. Incident hook mutates live road conditions.	PYTHON SIDECARS — non-real-time NLP chat + incident service; CV virtual-camera service — both WebSocket clients of the engine, own REST/WS endpoints.	OFFLINE PIPELINE — batch GA calibration optimizer; real-world CV congestion classifier; RL training → ONNX export. Feed static reports and trained policies.
--	--	---

Status: none of the four subjects exist in the codebase yet (m\ / is scaffolded but empty). This diagram shows the designed target architecture, detailed further in Part 4.

Explanation — How NexusSim Works

3.1 The simulation tick, step by step

Everything the engine does happens inside one repeating cycle called a *tick*. Every 0.1 simulated seconds, `Simulation::tick()` (`engine/src/agent/Simulation.h`) runs this sequence in order:

- **Step 1.** Advance simulation time; spawn any origin-destination-driven agents due this instant.
- **Step 2.** Tick every intersection's signal controller (Webster's-formula state machine: green → yellow → red → next phase).
- **Step 3.** Snapshot every active agent's position and velocity into thread-safe arrays.
- **Step 4.** Rebuild the quadtree spatial index from the snapshot.
- **Step 5.** Update all agents in parallel across CPU cores: compute IDM acceleration, query the quadtree for the nearest leading vehicle, check whether the next intersection is green, and probabilistically attempt a lane change.
- **Step 6.** Every 10th tick, recompute zone-level average wait time and the Gini coefficient across zones.
- **Step 7.** Broadcast the resulting state as one JSON frame over WebSocket.

Why the order matters. Steps 3 and 4 exist purely so step 5 can safely run on many CPU cores at once: the snapshot freezes a consistent copy of the world, so thousands of agents can be updated simultaneously without reading half-changed data.

3.2 Key algorithms

Five well-established algorithms do the heavy lifting. This table names each one, says what it does in practical terms, and gives the reason it was chosen over alternatives — useful for judging whether the simulation's behaviour is credible.

Table 5: The Five Core Algorithms and Why Each Was Chosen

Algorithm	What it does	Why this one was chosen
Intelligent Driver Model (IDM)	Computes each vehicle's acceleration from its own speed, the leading vehicle's speed, and the gap between them	Standard, well-validated car-following physics; parameterized per vehicle type (car / bus / rickshaw / two-wheeler / pedestrian)
A* pathfinding (+ stochastic variant)	Finds the shortest route from origin to destination; the stochastic variant adds lognormal noise to edge costs	The noisy variant approximates real route diversity — not every driver takes the mathematically optimal path
Quadtree spatial index	Answers “which agents are near this point” in roughly $O(\log n)$ instead of checking every other agent	Makes collision avoidance and lane-change checks tractable at 20,000+ concurrent agents
Webster's formula	Computes fixed-cycle green-light durations from expected traffic volume, once at startup	Classical, well-understood traffic-engineering baseline; the deliberate target for the RL replacement (Part 4)

Algorithm	What it does	Why this one was chosen
Gini coefficient	A single number (0 = perfectly equal, 1 = maximally unequal) summarizing how unevenly wait times are distributed across zones	Borrowed from economics; turns “is this policy fair?” into a reportable metric instead of a guess

KEY TAKEAWAYS

- Four of the five are **established, externally validated methods**, not custom inventions — which is what makes the simulation's behaviour defensible to a traffic engineer.
- Only one algorithm (**Webster's formula**) is explicitly framed as a baseline to be beaten; it is the single component the planned RL work replaces.
- The **quadtree** is the reason the 20,000-agent target in Part 4 is achievable at all — without it, every agent would have to check every other agent.

3.3 Typical workflow: running a scenario

Start to finish, using the system looks like this:

- **1.** Select or add a city in `cities.yaml` (query string, chaos coefficient, OD data source).
- **2.** Run the pipeline: `download` → `clean` → `infer lanes` → `tag zones` → `export graph.json`; `download` → `aggregate OD demand`.
- **3.** Launch the engine with that city's graph and OD matrix; it spawns agents at real hourly demand rates and starts ticking.
- **4.** Open the dashboard; watch live agents move on the map and metrics update every tick.
- **5.** Run `validate.py` to check simulated corridor journey times against real ground truth (checkpoint: $\text{MAPE} \leq 25\%$, $\geq 75\%$ of corridors within 25%).

3.4 Use case categories

Two broad classes of question NexusSim is designed to answer, expanded into concrete walkthroughs in Part 5:

- **“What happens if...”** — policy what-if testing (e.g. a bus lane, a signal-timing change) with before/after efficiency and equity metrics.
- **“React to this, right now”** — live scenario injection (e.g. a reported incident) and observing how the rest of the system (routing, signals, dashboard) responds.

Engineering Design

4.1 Key technical decisions

Six decisions shape how the system behaves. Each row states the choice made and the reasoning behind it. This matters because these are the trade-offs a reviewer would most likely challenge — each is recorded here rather than left implicit. Rows are grouped by whether the decision affects raw speed, or correctness and maintainability.

Table 6: Key Engineering Decisions and Their Rationale

Decision area	Chosen approach	Why this approach
PERFORMANCE-DRIVEN DECISIONS		
Agent memory layout	Structure-of-Arrays (parallel vectors per field), not array-of-structs	Better CPU cache utilization when updating thousands of agents per tick
Concurrency	std::async chunked across hardware threads, with a sequential snapshot phase first	Avoids data races without a full lock-based design; snapshot/update split keeps reads safe during parallel writes
CORRECTNESS & MAINTAINABILITY DECISIONS		
Transport	WebSocket + JSON (nlohmann::json)	Simple, debuggable, and the dashboard already tolerates unrecognized fields — extensible without breaking existing consumers
Signal timing baseline	Webster's formula, computed once at startup	Classical, well-understood; an explicit, honest baseline to beat rather than an unstated assumption
Validation methodology	Headless engine run + corridor-level MAPE against real trip data	Makes “is this realistic” a machine-checked number, not a subjective judgment
Per-city configuration	cities.yaml, not hardcoded constants	New cities added via config only (FR-6-equivalent requirement); already demonstrated with 3 cities at different data-quality levels

KEY TAKEAWAYS

- Only **2 of the 6 decisions** are made purely for speed; the other four buy debuggability, honesty about baselines, or the ability to add cities without code changes.
- The transport choice (plain JSON over WebSocket) deliberately trades some efficiency for **extensibility** — and Part 4.5 shows this is what makes the four-subject extension a non-breaking change.

4.2 Performance considerations

Four performance commitments, each with a specific number attached and a specific place in the codebase where it is measured. This matters because it shows performance is tracked continuously rather than asserted once.

Table 7: Performance Targets and How Each Is Measured

Concern	Target	Where it is measured	Why it matters
Agent capacity	20,000+ agents, no frame drops	engine/tests/test_stress.cpp	A city-scale simulation is only useful if it holds up at city-scale volume.
Proximity search speed	$\geq 10\times$ faster than brute force at 10,000 agents	engine/bench/bench_quadtree.cpp	Benchmarked directly against brute-force $O(n^2)$ search, so the gain is proven, not assumed.
Tick-time tracking	Average and p95 logged every 500 ticks	Simulation::avg_tick_ms(), p95_tick_ms()	Performance regressions become visible immediately, not discovered later.
Network payload	Bounded regardless of city size	Level-of-detail viewport culling	The dashboard sends its viewport bounds; agents outside that view are omitted from the broadcast.

KEY TAKEAWAYS

- Every target is a **number with a test behind it** — there are no unmeasured performance claims in this document.
- The **10× quadtree speedup** is measured against the naive alternative, which is the only comparison that proves the added complexity earns its place.
- Payload size is controlled by what the user can actually see, so a larger city does not mean a slower dashboard.

Presentation note: revision 1 stated these four items as a prose bullet list. The values, file paths, and targets are unchanged — only the layout is now tabular, and the “Why it matters” column restates reasoning already present in the original text.

4.3 Scalability path

Growth can come from three different directions, and each is handled differently on purpose. This table separates what already works today from what is designed but not yet built.

Table 8: How the System Scales, by Growth Dimension

Growth dimension	Current answer (built today)	Planned answer (designed, not yet built)
More agents in one city	Quadtree + thread pool already handle 20,000+	Same architecture; no redesign needed at currently targeted scale
More cities	Config-driven (cities.yaml); 3 cities declared today, 1 fully wired	Same pattern extends to new cv_bbox / signal_policy_default keys for the four-subject extension

Growth dimension	Current answer (built today)	Planned answer (designed, not yet built)
Real-time ML inference	Not applicable yet (no ML in the loop)	Offline training in Python, ONNX Runtime inference in C++, batched across intersections, target $p95 \leq 8$ ms per tick — explicitly designed so a live Python process is never in the simulation's hot path

KEY TAKEAWAYS

- **Two of three dimensions need no new engineering** — more agents and more cities are already handled by the current design.
- Only the third (real-time AI inference) requires genuinely new work, and its design constraint is explicit: a **p95 budget of 8 ms per tick**, well inside the 0.1 s tick.
- Of 3 declared cities, only 1 is fully wired today — the clearest gap between what is configured and what is proven.

4.4 Planned engineering additions (per-subject)

Reinforcement Learning

Independent agent per signalized intersection, trained offline (PyTorch, one agent's decision = extend current phase or switch to the next), against a shared reward combining local wait time with network-wide average wait time and the Gini coefficient — both already computed by the engine today. Exported to ONNX; runtime inference happens in C++, never blocking the 0.1 s tick budget.

Computer Vision

Two independent pipelines: (1) classify real-world congestion from traffic-tile imagery (Mapbox or TomTom — not Google's Static Maps API, which exposes no scriptable live-traffic layer), compared classical-threshold vs. CNN; (2) a synthetic virtual camera that renders the live simulation from a top-down view and runs real OpenCV detection/counting on the rendered frame.

NLP

A rule-based intent classifier (with an optional LLM layer for comparison) answers questions about live metrics; a separate parser resolves free-text incident reports against a street-name gazetteer built from the graph itself, and applies a temporary capacity/speed penalty to the affected road segments in the running simulation.

Soft Computing

A genetic algorithm replaces the manual parameter search currently in `validate.py`, evolving (`speed_factor`, `route_spread`, `chaos`, `demand_scale`) against the existing MAPE checkpoint as fitness. A fuzzy-logic signal controller provides a third, non-learned strategy directly comparable against Webster's formula and the RL policy.

4.5 Design principle: additive. not disruptive

Every planned addition in §4.4 is designed to extend the existing interfaces (WebSocket JSON, `cities.yaml`, the per-intersection signal-controller slot) rather than replace them. The dashboard's WebSocket hook already ignores JSON fields it doesn't recognize, which is precisely what makes adding new broadcast fields (signal mode, active incidents) a non-breaking change instead of a protocol redesign.

Examples & Use Cases

Three worked walkthroughs, ordered from what works today to what is planned. Each is laid out as a numbered sequence so the flow of cause and effect is easy to follow.

Use Case 1 — Testing a Bus Lane Policy (built today)

Scenario: A city planner wants to know whether dedicating one lane on a congested corridor to buses only would help overall commute equity, without repainting a single real road stripe to find out.

Table 9: Bus Lane Policy Test — Step-by-Step Walkthrough

Step	What happens	What it produces
1. Baseline run	Engine runs Chicago with real OD demand and default lane configuration.	avg_wait_time and the Gini coefficient, recorded from the live broadcast.
2. Policy change	The corridor's lane allocation is adjusted for the bus agent type; the simulation is re-run with identical OD demand.	A second set of metrics directly comparable to the baseline.
3. Compare	The dashboard's efficiency and equity panels show the before/after delta.	An answer to: did average wait time improve, and did it improve evenly across zones (lower Gini) or only in already-well-served ones?
4. Validate realism	validate.py confirms the baseline run still meets the $\text{MAPE} \leq 25\%$ checkpoint against real Chicago corridor data.	Confidence that the comparison is trustworthy, not just internally consistent.

KEY TAKEAWAYS

- Step 2 holds demand **identical** to step 1 — without that, the before/after comparison would be meaningless.
- Step 4 is what separates this from a plausible-looking demo: the baseline must still match real Chicago data to within **25% MAPE** before any conclusion is drawn.
- This is the core, already-implemented use case the whole engine, pipeline, and dashboard exist to support (docs/PRD.md §1–§2).

Use Case 2 — Incident Report Reacting Live (planned, cross-subsystem)

Scenario: An operator types “accident on Michigan Ave, lane closure” into the dashboard's incident box while a simulation is running with the RL signal policy active. This is the clearest demonstration that the four extension subjects form one system rather than four separate demos (Part 2.4).

1. NLP incident parser Resolves “Michigan Ave” against the graph's street names, builds a structured incident spec	2. Engine (WebSocket) Applies a temporary speed/capacity penalty to the affected road edges	3. RL policy + Dashboard Nearby intersections' queue observations shift automatically; the incident and its effect appear live on the map
--	---	---

Table 10: Live Incident Injection — Step-by-Step Walkthrough

Step	What happens	Subsystem responsible
1. Text submitted	“Accident on Michigan Ave, lane closure” sent to the NLP incident endpoint.	NLP
2. Parsing	Fuzzy string match against the street-name gazetteer built from graph.json edge names resolves the affected edge IDs; a small vocabulary maps “accident” + “lane closure” to a severity and default duration.	NLP
3. Simulation mutation	The engine's apply_incident() reduces speed/capacity on the affected edges for the incident's duration, then automatically reverts.	Engine
4. Automatic reaction	No incident-specific code exists in the RL policy — it reacts because its normal observations (queue length near the closure) change, exactly as it would to any other real congestion.	Reinforcement Learning
5. Visibility	The dashboard's incident panel shows the active incident and its remaining duration; nearby zone metrics visibly shift in the equity/efficiency panels.	Dashboard

KEY TAKEAWAYS

- Step 4 is the whole point: the RL policy needs **no special code** for incidents. One text box changes the road network, and the AI adapts on its own.
- Four different subsystems participate in a single user action, which is what makes this the document's strongest evidence of genuine integration rather than four parallel demos.
- The incident **reverts automatically** after its parsed duration — no manual cleanup step.

Use Case 3 — Comparing Signal Strategies (planned)

Scenario: A technical reviewer wants to see, side by side, whether the learned RL signal policy actually beats the classical baseline — and by how much, and for whom.

Table 11: Signal Strategy Comparison — Step-by-Step Walkthrough

Step	What happens	Why this step exists
1. Common ground	Same city, same OD demand, same GA-tuned calibration parameters for every run.	Removes every variable except the signal strategy itself.
2. Three runs	Webster's formula, a fuzzy-logic controller, and the trained RL/ONNX policy are each run in turn — or toggled live in the same running simulation via a <code>policy_switch</code> message, with no restart.	Live toggling makes the comparison directly observable rather than a table of numbers from separate sessions.
3. Metrics captured	Average city-wide wait time and the Gini coefficient across zones, for each strategy.	Captures both halves of the objective: speed and fairness.
4. Reported result	A comparison table (and, for the RL policy specifically, an efficiency-vs-equity tradeoff curve as the reward's fairness weight is varied).	Makes the tradeoff a measured, visible decision rather than a single blindly maximized number.

KEY TAKEAWAYS

- **Three strategies** are compared, not two — the fuzzy-logic controller sits between the classical baseline and the learned policy, showing how much gain comes from adaptivity versus from learning specifically.
- Both metrics are always reported **together**: a strategy that improves speed while worsening the Gini coefficient is not counted as a win.

NexusSim — Explained, Engineered (revision 2) · tables and explanations rebuilt for readability; no data values altered · companion to IMPLEMENTATION_PLAN.md, PRD.md, TECH_STACK.md, and docs/NexusSim_Explained.md